

Höldum áfram í næsta skref. Búum til "v0.6 — Application Architecture & Implementation Specification".

Frábært. Nú færum við okkur úr gagnagrunns- og workflow-hönnun yfir í **raunverulega application architecture**. Hér er v0.6 sem implementation blueprint fyrir kerfið.

Technical Specification v0.6

Application Architecture & Implementation Specification

Project: Icelandic Legal Case Management System — ILCMS

Version: 0.6

Status: Architecture / Implementation Baseline

Date: September 2026

1. Purpose

Technical Specification v0.6 skilgreinir application architecture fyrir ILCMS og tengir saman:

- Web application
- REST API
- Authentication & authorization
- Application/service layer
- PostgreSQL
- Document/object storage
- Background workers
- Search
- Notifications
- Audit
- AI services
- Observability
- Deployment

Markmiðið er að architecture sé:

- secure by design
- modular

- auditable
- scalable
- testable
- suitable for on-premises eða private-cloud deployment
- möguleg að keyra air-gapped síðar
- tilbúin fyrir AI integration án þess að veita AI óheftan aðgang að málaskjöllum.

2. Architecture Principles

A1 — API-first

Öll business functionality skal vera aðgengileg í gegnum application service/API layer.

Frontend má ekki tala beint við PostgreSQL.

Browser



API



Services



Repositories



PostgreSQL

A2 — Zero Trust

Hver request skal sannprófaður.

Authentication



Authorization



Case access



Resource access



Action

Ekki skal treysta á að notandi sé öruggur eingöngu vegna þess að hann hefur rétt role.

A3 — Least Privilege

Hver component fær aðeins þau permissions sem hann þarf.

Dæmi:

Web

→ API only

API

→ PostgreSQL

→ Object storage

→ Queue

OCR worker

→ Object storage READ

→ OCR output WRITE

→ PostgreSQL limited access

A4 — Immutable originals

Original skjöl og mikilvægar audit upplýsingar skulu vera immutable.

A5 — Async processing

Pungar aðgerðir skulu ekki keyra inni í HTTP request:

- OCR
- malware scanning
- large exports
- indexing
- notifications
- AI processing

A6 — Authoritative data

PostgreSQL er authoritative source fyrir:

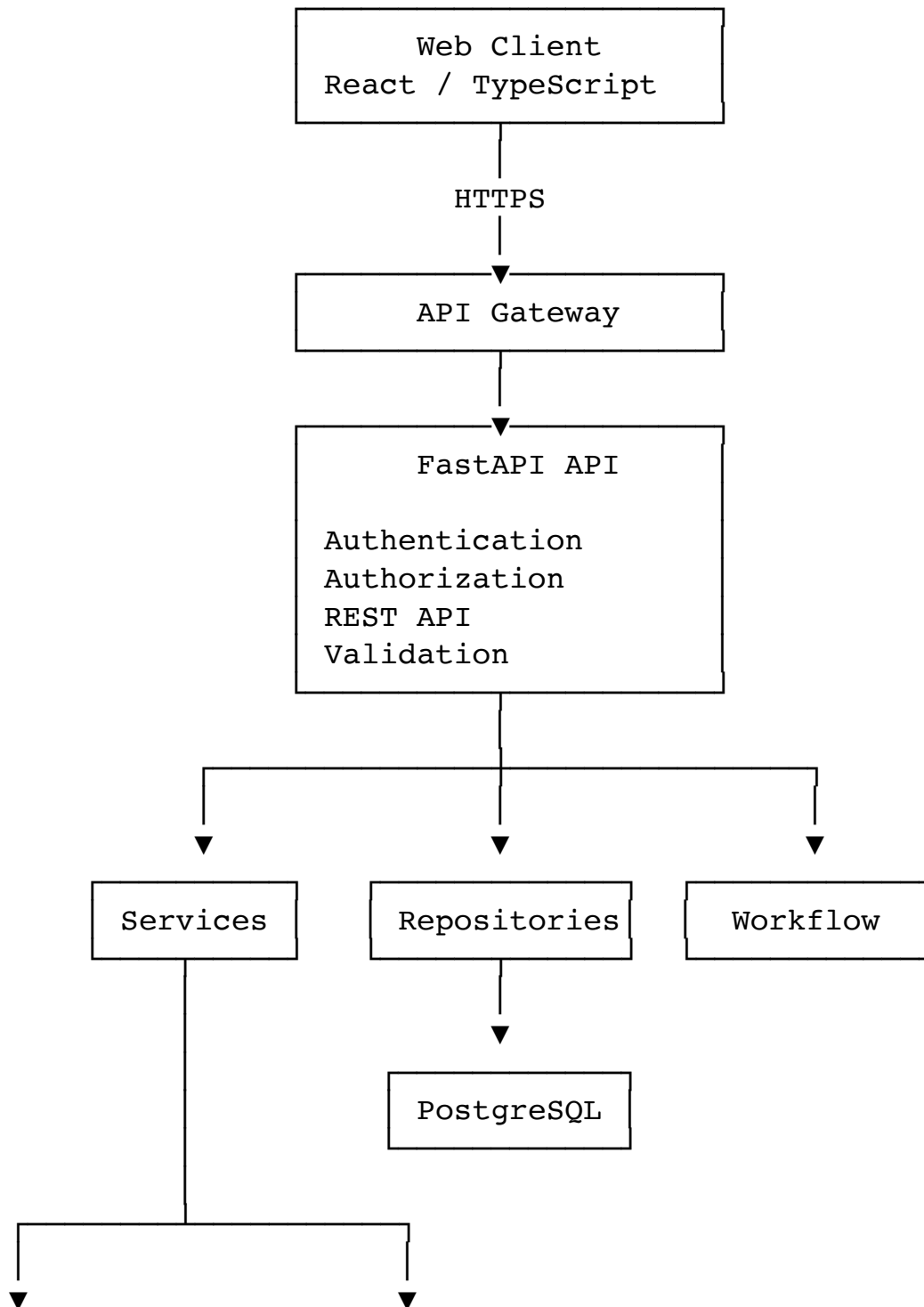
- cases
- users
- permissions
- workflow

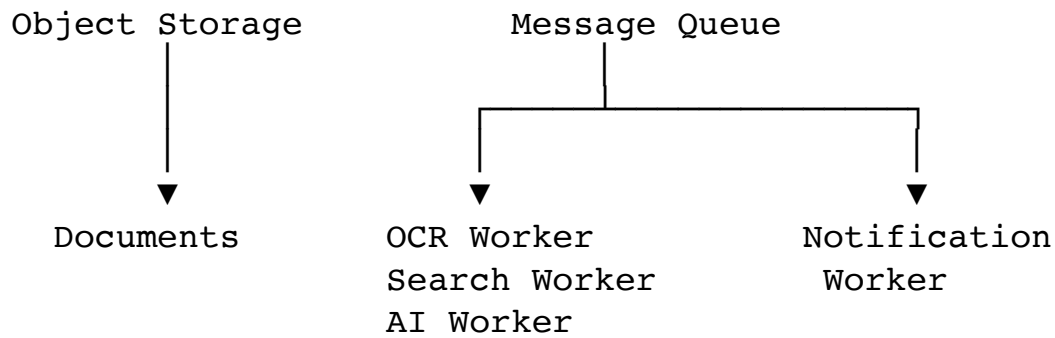
- metadata
- audit references

Object storage er authoritative source fyrir document binaries.

Search index er **ekki** authoritative source.

3. Logical Architecture





4. Recommended Technology Stack

4.1 Frontend

React + TypeScript

Recommended:

React

TypeScript

Vite

React Router

TanStack Query

Optional:

Zustand

React Hook Form

UI component framework can be selected separately.

5. Backend

Python + FastAPI

Reasoning:

- strong type support
- excellent REST API support
- async support
- Pydantic validation
- mature PostgreSQL ecosystem
- easy integration with OCR/AI/ML libraries
- good testing ecosystem.

Recommended:

Python 3.12+
FastAPI
Pydantic
SQLAlchemy 2.x
Alembic
psycopg

6. Application Layers

Backend skal skiptast í:

```
API Layer
  ↓
Application / Service Layer
  ↓
Domain Layer
  ↓
Repository Layer
  ↓
Infrastructure
```

7. API Layer

API layer sér um:

- HTTP
- authentication context
- request validation
- response serialization
- API errors
- rate limiting
- OpenAPI documentation.

API layer á ekki að innihalda complex business logic.

Dæmi:

```
@router.post("/cases")
def create_case(
    request: CreateCaseRequest,
```

```
        current_user: CurrentUser
    ):
        return case_service.create_case(
            request,
            current_user
        )
```

8. Service Layer

Service layer sér um business operations.

Dæmi:

```
CaseService
DocumentService
EvidenceService
FilingService
DeadlineService
ApprovalService
RetentionService
PermissionService
SearchService
```

Dæmi:

```
DocumentService.upload()
```

↓

```
authorize_upload()
```

↓

```
store_object()
```

↓

```
create_document_version()
```

↓

```
create_audit_event()
```

↓

```
publish_outbox_event()
```

9. Domain Layer

Domain layer skilgreinir:

- entities
- business rules
- state transitions
- invariants
- domain events.

Dæmi:

Case

Document

Evidence

CourtFiling

LegalHold

Deadline

Case state transition skal vera domain operation:

```
case.transition_to(LITIGATION)  
en ekki:
```

```
case.status = "LITIGATION"
```

10. Repository Layer

Repository sér um persistence.

Dæmi:

CaseRepository

DocumentRepository

EvidenceRepository

UserRepository

AuditRepository

Service layer veit ekki nákvæmlega hvernig SQL er keyrt.

Service

↓

Repository

↓

SQLAlchemy

↓

PostgreSQL

11. Infrastructure Layer

Infrastructure inniheldur adapters fyrir:

PostgreSQL

Object Storage

Message Queue

Email

Identity Provider

Search

OCR

AI

Dæmi:

S3ObjectStorage

PostgresCaseRepository

RabbitMQEventBus

OpenSearchAdapter

KeycloakIdentityProvider

Þetta gerir infrastructure replaceable.

12. Authentication

ILCMS skal ekki geyma password hashes sem primary identity mechanism í production ef hægt er að nota enterprise identity provider.

Recommended:

OIDC / OAuth 2.0

Architecture:

Browser

↓

Identity Provider

↓

Access Token

↓

ILCMS API

↓

Validate token

Mögulegir identity providers:

Microsoft Entra ID

Keycloak

Okta

Auth0

Fyrir íslenskt lögfræðikerfi er **Keycloak** áhugaverður kostur ef on-prem/private deployment og sjálfstæði frá cloud er mikilvægt.

13. Authentication vs Authorization

Þetta verður að vera skýrt:

Authentication

= Who are you?

Authorization

= What may you do?

Token segir hver notandinn er.

ILCMS authorization layer ákveður síðan:

Can user X

perform action Y

on resource Z?

14. Authorization service

Central authorization component:

AuthorizationService

Dæmi:

```
authorize(  
    user,  
    action="DOCUMENT:EXPORT",  
    resource=document  
)
```

Authorization framkvæmir:

Role permission

+

Case ACL

+

Confidentiality

+

Privilege

+

Workflow state
+
Legal hold

15. API Structure

Base path:

/api/v1

Dæmi:

```
GET      /cases
POST     /cases
GET      /cases/{case_id}
PATCH   /cases/{case_id}

POST     /cases/{case_id}/transitions

GET      /cases/{case_id}/documents
POST     /cases/{case_id}/documents

GET      /documents/{document_id}
GET      /documents/{document_id}/versions

POST     /documents/{document_id}/download
POST     /documents/{document_id}/export

GET      /cases/{case_id}/evidence
POST     /cases/{case_id}/evidence

GET      /cases/{case_id}/deadlines
POST     /cases/{case_id}/deadlines
```

16. API Error Model

Öll API errors skulu hafa staðlað format:

```
{
  "error": {
    "code": "DOCUMENT_ACCESS_DENIED",
    "message": "Access denied.",
    "request_id": "..."
  }
}
```

Ekki skila security-sensitive upplýsingum:

"User lacks HIGHLY_CONFIDENTIAL permission"
Beta:

"Access denied."

17. Request correlation

Hver HTTP request fær:

request_id

Dæmi:

HTTP request

↓

request_id = abc123

↓

API log

↓

service log

↓

database audit

↓

worker event

Þetta gerir incident investigation mun auðveldari.

18. Document Upload Architecture

Browser

|

multipart / presigned upload

▼

API

|

Authorization

Upload session

|

▼

Object Storage

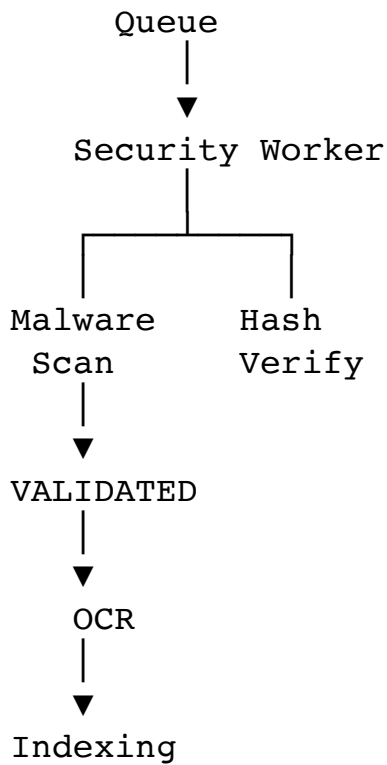
|

▼

Upload Event

|

▼



19. Presigned Uploads

Stór skjöl ættu ekki endilega að fara í gegnum FastAPI process.

Betra:

```
Browser
  ↓
POST /documents/upload-session
  ↓
API authorizes
  ↓
Presigned URL
  ↓
Browser → Object Storage
API heldur áfram að hafa fulla stjórn á authorization.
```

20. Malware scanning

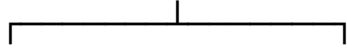
Öll uploaded files skulu fara í quarantine fyrst.

```
UPLOAD
  ↓
QUARANTINE
  ↓
```

MALWARE SCAN



CLEAN?



YES

NO



VALIDATED

QUARANTINED

Ekki leyfa document download úr quarantine nema privileged security operation.

21. OCR

OCR skal vera asynchronous.

Document



OCR Job



Worker



OCR Engine



Extracted text



PostgreSQL / Search index

Original binary breytist aldrei.

22. Search architecture

MVP:

PostgreSQL Full Text Search

Scale-up:

PostgreSQL



Outbox



Search Worker



OpenSearch

Search result verður síðan filtered eftir authorization.

Aldrei:

Search first
Authorization later
Better:

Authorized scope

↓

Search

↓

Results

eða security-filtered search index.

23. Search Security

Critical requirement:

Search má ekki leka existence of unauthorized documents.

Dæmi:

User A má ekki fá:

"No results because document X is confidential."

Ef document X er unauthorized ætti það að hegða sér eins og það sé ekki til.

24. Background Jobs

Queue-based architecture:

API

↓

Outbox

↓

Message Broker

↓

Workers

Jobs:

document.scan

document.ocr

document.index

document.export

notification.send

deadline.reminder

retention.evaluate

ai.summarize

25. Message Broker

Recommended options:

MVP

RabbitMQ

Simpler deployment

Redis

Cloud

Azure Service Bus

AWS SQS

Google Pub/Sub

Fyrir portable/private deployment er **RabbitMQ** góður default.

26. Outbox Pattern

API skal ekki:

INSERT database

+

send message

án consistency mechanism.

Nota:

BEGIN

INSERT document

INSERT outbox_event

COMMIT

Síðan:

Outbox Worker

↓

Message Broker

Þetta kemur í veg fyrir að database update gerist en event tapist.

27. Event model

Dæmi:

```
{
  "event_id": "...",
  "event_type": "DocumentValidated",
  "aggregate_type": "document",
  "aggregate_id": "...",
  "occurred_at": "...",
  "payload": {
    "case_id": "...",
    "document_id": "..."
  }
}
```

28. Idempotency

Workers skulu vera idempotent.

Ef:

`document.ocr`

kemur tvisvar má ekki skapa tvö conflicting OCR results.

Worker skal geta gert:

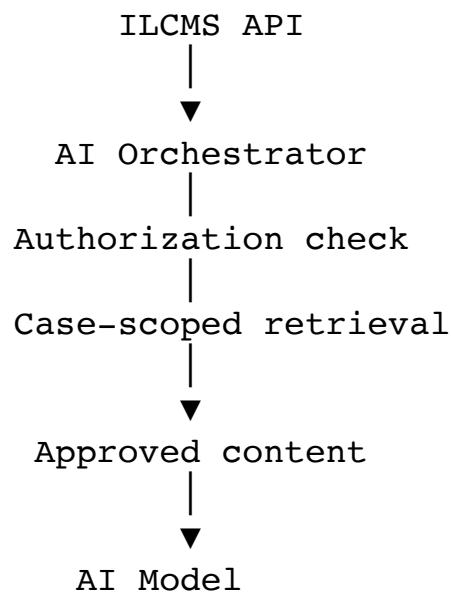
same event

→ same final state

eða safely detect duplicate processing.

29. AI Architecture

AI verður sérstakt security boundary.



AI fær aldrei:

`DATABASE_PASSWORD`
og helst ekki beinan database access.

30. AI capabilities

Mögulegar MVP+ features:

Document summarization
Case summarization
Semantic search
Document classification
Entity extraction
Timeline generation
Deadline extraction
Legal issue extraction

31. AI Guardrails

AI service skal aðeins fá:

authorized documents
og ekki:

all documents belonging to organization
Niðurstaða AI skal vera merkt sem:

`AI_GENERATED`
og ekki sjálfkrafa teljast staðfest lögfræðileg niðurstaða.

32. AI Human Review

AI output:

GENERATED

↓

REVIEWED

↓

ACCEPTED / REJECTED

Ekki:

AI

↓

Automatic legal decision

33. Object Storage

S3-compatible object storage er recommended abstraction.

Möguleikar:

AWS S3

Azure Blob

MinIO

Ceph

Application notar abstraction:

```
object_storage.put(...)
```

```
object_storage.get(...)
```

```
object_storage.delete(...)
```

Pannig er hægt að skipta provider síðar.

34. Storage separation

Mælt:

documents-original

documents-derived

documents-quarantine

documents-export

backups

Original objects skulu hafa strangari retention policy.

35. Encryption keys

Key management skal vera aðskilið frá application secrets.

Architecture:

Application

↓

KMS / Vault

↓

Encryption keys

Fyrir private/on-prem:

HashiCorp Vault

eða HSM/KMS lausn eftir security requirements.

36. Audit architecture

Audit skal gerast bæði á application level og critical database level.

Application

↓

Audit Service

↓

Audit DB

Critical DB changes

↓

Database audit mechanisms

Audit event:

```
{  
  "action": "DOCUMENT_EXPORT",  
  "user_id": "...",  
  "case_id": "...",  
  "document_id": "...",  
  "timestamp": "...",  
  "request_id": "..."  
}
```

37. Logging

Logs mega ekki innihalda:

passwords

access tokens

document contents

private keys

secrets

Varast einnig að logga:

full legal documents

full AI prompts

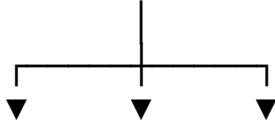
sensitive PII

nema explicit policy leyfi það.

38. Observability

Recommended:

OpenTelemetry



Logs Metrics Traces

Metrics:

API latency

API errors

queue depth

worker failures

OCR duration

document processing time

database connections

storage failures

39. Monitoring

Infrastructure:

Prometheus

Grafana

Logs:

Loki

eða:

OpenSearch

Tracing:

OpenTelemetry

40. Health endpoints

GET /health/live

GET /health/ready

live:

Process is alive.

ready:

Database available

+

required dependencies available

Ekki setja sensitive diagnostic upplýsingar í public response.

41. Configuration

Configuration skal vera environment/configuration driven.

Dæmi:

```
DATABASE_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OIDC_ISSUER
OIDC_CLIENT_ID
QUEUE_URL
SEARCH_ENDPOINT
VAULT_ADDR
Secrets koma úr secret manager.
```

Ekki Git.

42. Containerization

Allir application components skulu vera containerizable.

```
ilcms-api
ilcms-worker
ilcms-search-worker
ilcms-ocr-worker
ilcms-ai-worker
ilcms-frontend
```

43. Kubernetes deployment

Production architecture getur verið:

```
Kubernetes
├── frontend
├── api
├── workers
├── postgres
├── rabbitmq
├── opensearch
└── monitoring
```

En PostgreSQL/OpenSearch mega einnig vera managed services.

44. MVP Deployment

Til að forðast óþarfa complexity:

Docker Compose

↓

Development / Test

og:

Kubernetes

↓

Production

Architecture skal vera sú sama.

45. Environments

Að minnsta kosti:

DEV

TEST

STAGING

PRODUCTION

Gögn skulu ekki flytjast beint:

PRODUCTION → DEV

nema þau séu anonymized/sanitized.

46. CI/CD

Recommended:

Git

↓

Pull Request

↓

Lint

↓

Unit Tests

↓

Security Scan

↓

Build Container

↓

Integration Tests

↓

Migration Test

↓

Deploy STAGING

↓

Approval

↓

Deploy PRODUCTION

47. Dependency security

CI skal keyra:

SAST

Dependency scanning

Container scanning

Secret scanning

SBOM generation

Möguleg tools:

Semgrep

Trivy

Syft

Gitleaks

Dependabot / Renovate

48. Database migrations

Migration flow:

Developer

↓

Alembic migration

↓

CI

↓

Test database

↓

Staging

↓

Production

Production migration skal vera explicit deployment step.

49. Database connection security

API:

TLS → PostgreSQL

Connection pooling:

FastAPI

↓

SQLAlchemy pool

↓

PostgreSQL

Pool size skal stilla eftir fjölda API replicas.

Ekki:

100 replicas × 50 DB connections
án capacity planning.

50. API rate limiting

Rate limiting eftir:

user

client

IP

endpoint

Sérstaklega fyrir:

login

search

export

AI requests

file upload

51. File download security

Ekki skila permanent public URL.

Betra:

GET /documents/{id}/download

↓

Authorization

↓

Short-lived signed URL

↓

Object storage

eða streamed response fyrir sérstaklega viðkvæm gögn.

52. Export security

Export:

Request

↓

Authorization

↓

Optional approval

↓

Generate package

↓

Hash

↓

Audit

↓

Short-lived download

Package skal hafa manifest.

53. Frontend Security

Frontend má aldrei treysta eigin authorization logic.

Til dæmis:

Hide "Export" button

er UX, ekki security.

Backend þarf samt:

DOCUMENT:EXPORT

authorization.

54. Frontend modules

Dashboard

Cases

└─ Overview

- Parties
- Documents
- Evidence
- Issues
- Facts
- Arguments
- Proceedings
- Deadlines
- Audit

Documents
 Search
 Tasks
 Approvals
 Administration

55. Case workspace

Aðal UI hugmynd:

CASE LGM-2026-00123	
Title	
Overview Parties Docs Evidence Issues Facts Court Deadlines Audit	Case Workspace

56. Security-aware UI

Frontend skal fá permission information:

```
{
  "permissions": [
    "CASE:READ",
    "DOCUMENT:READ",
    "DOCUMENT:DOWNLOAD"
  ]
}
```

}

Þetta gerir UI kleift að fela óviðeigandi actions.

Backend er samt authoritative.

57. Testing Architecture

Testing levels:

Unit

↓

Integration

↓

API

↓

Security

↓

End-to-End

↓

Performance

58. Security tests

Sérstaklega:

Unauthorized case access

Cross-case document access

Privilege escalation

Role bypass

Export without permission

Archived case modification

Legal hold bypass

Audit manipulation

Search leakage

AI cross-case leakage

59. Example security test

User A

↓

Case A = authorized

Case B = unauthorized

Request:
GET /cases/B/documents

Expected:
403
Ekki:

200 + empty list
nema API security policy krefjist þess sérstaklega.

60. Performance targets

Initial design targets:

Normal API request:
< 300 ms p95

Database query:
< 100 ms p95

Document metadata retrieval:
< 300 ms p95

Search:
< 1 second p95

Async document processing:
system dependent
Large document upload og OCR eru ekki hluti af normal HTTP latency target.

61. Availability

Initial target:

99.9% monthly availability
Critical components:

API
PostgreSQL
Object Storage
Queue
Identity Provider

62. Disaster Recovery

Target:

RPO $\leq$ 15 minutes

RTO $\leq$ 1 hour

Backup:

PostgreSQL

Object storage

Configuration

Secrets metadata

Search indexes

Search index má vera reconstructable.

63. Disaster recovery principle

Ef OpenSearch tapast:

PostgreSQL

↓

Outbox / reindex job

↓

OpenSearch rebuilt

Ef object storage tapast:

Backup

↓

Restore

Documents mega ekki vera aðeins varðveitt í search index.

64. Data classification

Architecture skal styðja:

PUBLIC

INTERNAL

CONFIDENTIAL

HIGHLY_CONFIDENTIAL

PRIVILEGED

Þetta þarf að endurspeglast í:

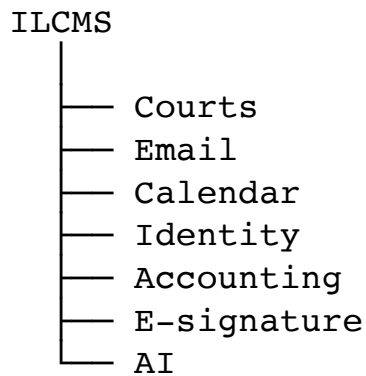
Database

Object storage

Search
Logs
Backups
AI
Exports

65. External integrations

Future integration layer:

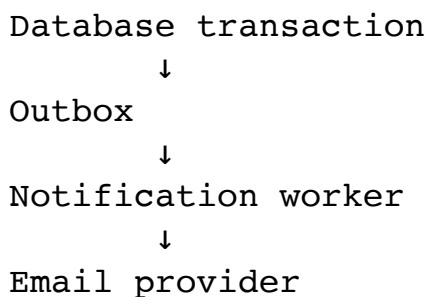


Integrations skulu fara í gegnum adapters.

Ekki hard-code external API calls inn í domain logic.

66. Email architecture

Ekki senda email beint úr transaction.



67. Calendar integration

Deadlines geta síðar tengst:

Microsoft 365
Google Workspace
CalDAV
En ILCMS deadline verður authoritative.

External calendar er projection.

68. E-signature

Potential future:

Document

↓

Approval

↓

E-signature provider

↓

Signed document

↓

New document version

↓

Audit

Signing má ekki breyta original document.

69. API versioning

/api/v1

Breaking changes:

/api/v2

Ekki gera breaking changes innan v1.

70. OpenAPI

FastAPI skal sjálfkrafa framleiða:

OpenAPI

Swagger UI

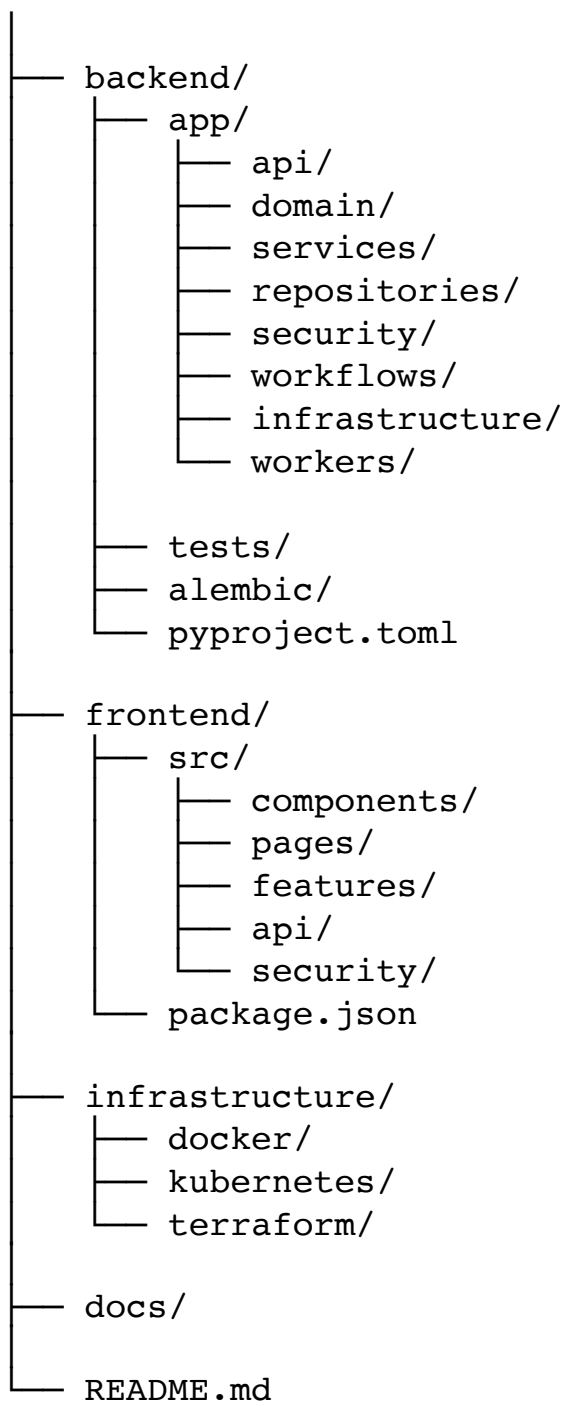
ReDoc

API specification skal vera hluti af source control.

71. Repository structure

Recommended:

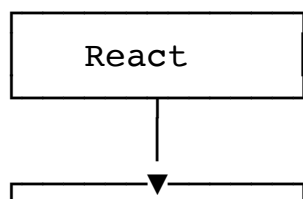
ilcms/

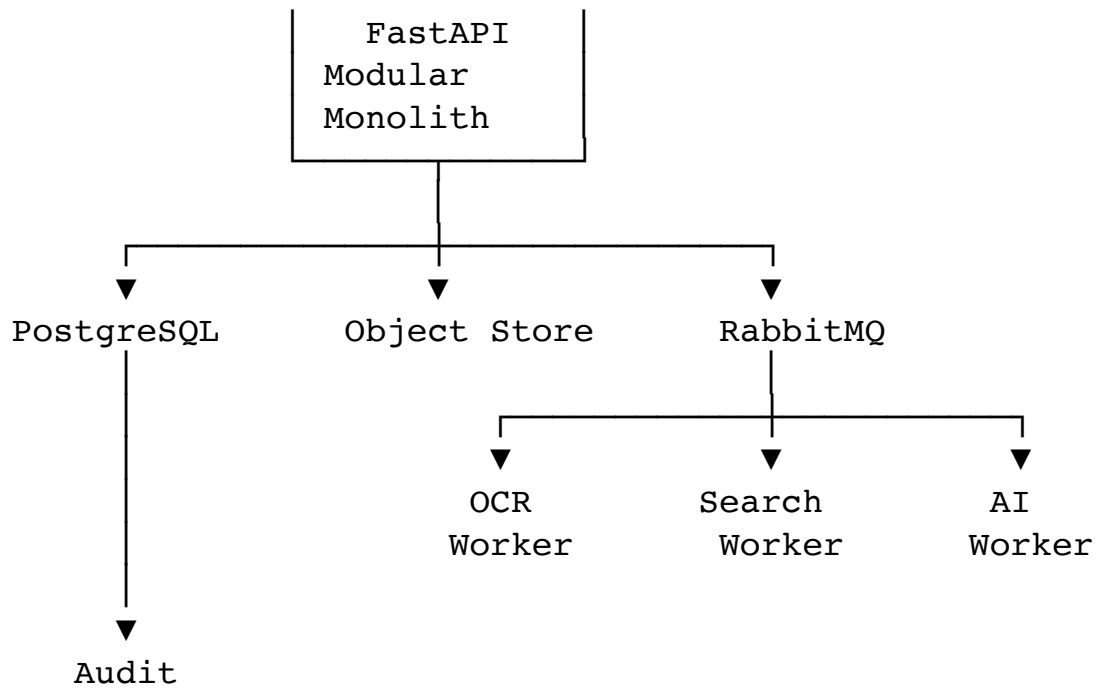


72. Recommended MVP Components

Fyrsta útgáfa ætti ekki að vera microservice zoo.

Mælt:





73. Modular Monolith

Þetta er mikilvæg architecture decision.

Ég myndi **ekki** byrja með:

15 microservices

Heldur:

One FastAPI application

+

Modular domain architecture

+

Independent workers

Þetta gefur okkur:

- einfaldari deployment
- einfaldari debugging
- einfaldari transactions
- minna operational overhead
- samt skýra boundaries.

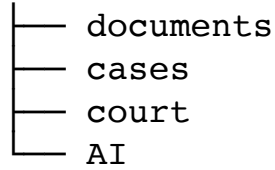
Seinna má extract-a module í service ef þörf verður á.

74. Service extraction path

Til dæmis:

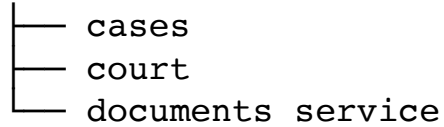
v1

FastAPI



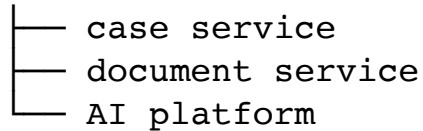
v2

FastAPI



v3

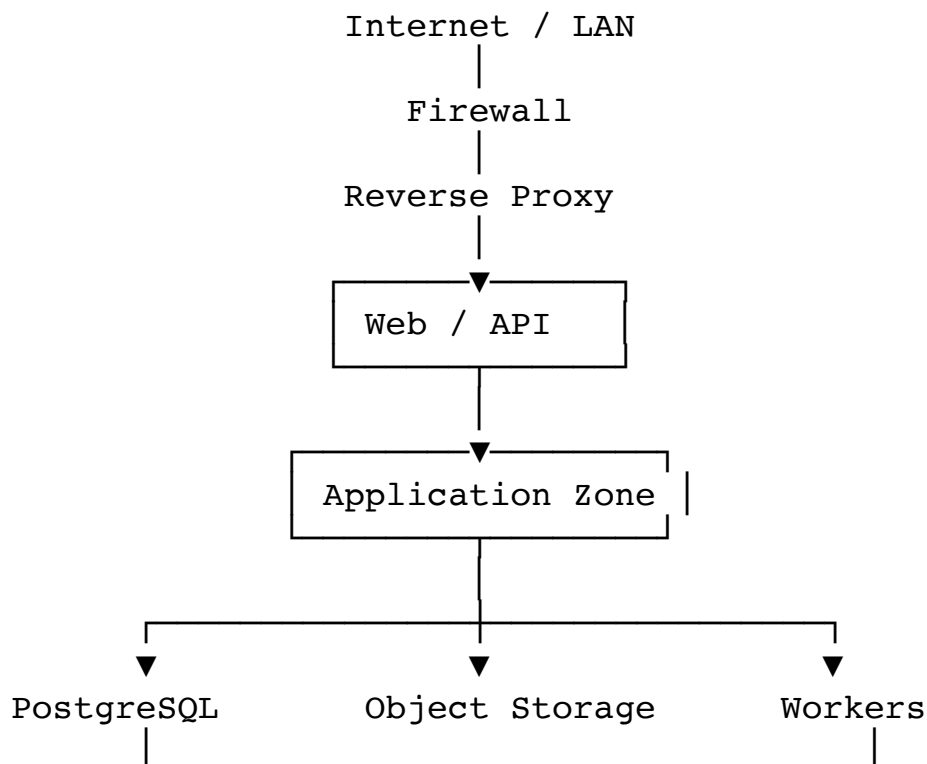
FastAPI

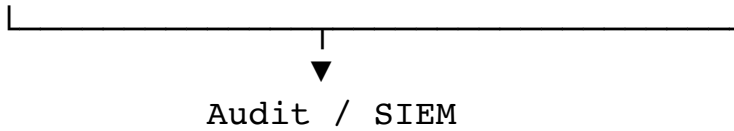


Architecture má því þróast án þess að byrja of flókið.

75. Security Boundary Model

Production architecture:





76. Air-gapped compatibility

Architecture skal ekki gera ráð fyrir Internet access.

All external dependencies skulu vera replaceable.

Þetta þýðir að hægt sé að keyra:

Private Kubernetes
+
Private PostgreSQL
+
MinIO
+
RabbitMQ
+
OpenSearch
+
Keycloak
+
Local LLM
án Internet.

Þetta er sérstaklega mikilvægt ef kerfið síðar verður notað fyrir mjög viðkvæm lögfræðileg gögn.

77. Local AI architecture

Air-gapped AI:

ILCMS
↓
AI Gateway
↓
Local Model Server
↓
LLM
Possible future model server:

vLLM
Ollama
llama.cpp

Model data verður innan trusted environment.

78. AI data boundary

Strict rule:

CASE A

↓

Authorized retrieval

↓

AI context

↓

AI response

Ekki:

All cases

↓

LLM context

Petta skal vera architectural invariant.

79. MVP Scope

Include

Authentication

Cases

Parties

Documents

Document versions

Evidence

Issues

Facts

Legal arguments

Court proceedings

Court filings

Deadlines

RBAC

Case ACL

Audit

Retention

Legal holds

Search

Notifications

Phase 2

Advanced OCR
Advanced search
E-signatures
External integrations
AI summarization
Semantic search

Phase 3

AI agents
Advanced legal research
Predictive analytics
Workflow automation
External portals

80. Architecture Decision Record

ADR-001 — Modular Monolith

Decision: Use modular monolith for initial application.

Reason: Lower operational complexity while retaining clean domain boundaries.

ADR-002 — PostgreSQL

Decision: PostgreSQL 16+.

Reason: Strong transactional consistency, mature ecosystem, JSONB, full-text search and excellent support for relational legal data.

ADR-003 — Object Storage

Decision: S3-compatible abstraction.

Reason: Large binaries should not be stored directly in PostgreSQL.

ADR-004 — Async Workers

Decision: RabbitMQ + worker processes.

Reason: Document processing and external integrations should not block API requests.

ADR-005 — OIDC

Decision: OIDC-based authentication.

Reason: Enterprise identity integration and reduced password-management burden.

ADR-006 — AI Isolation

Decision: AI operates through an authorization-aware AI gateway.

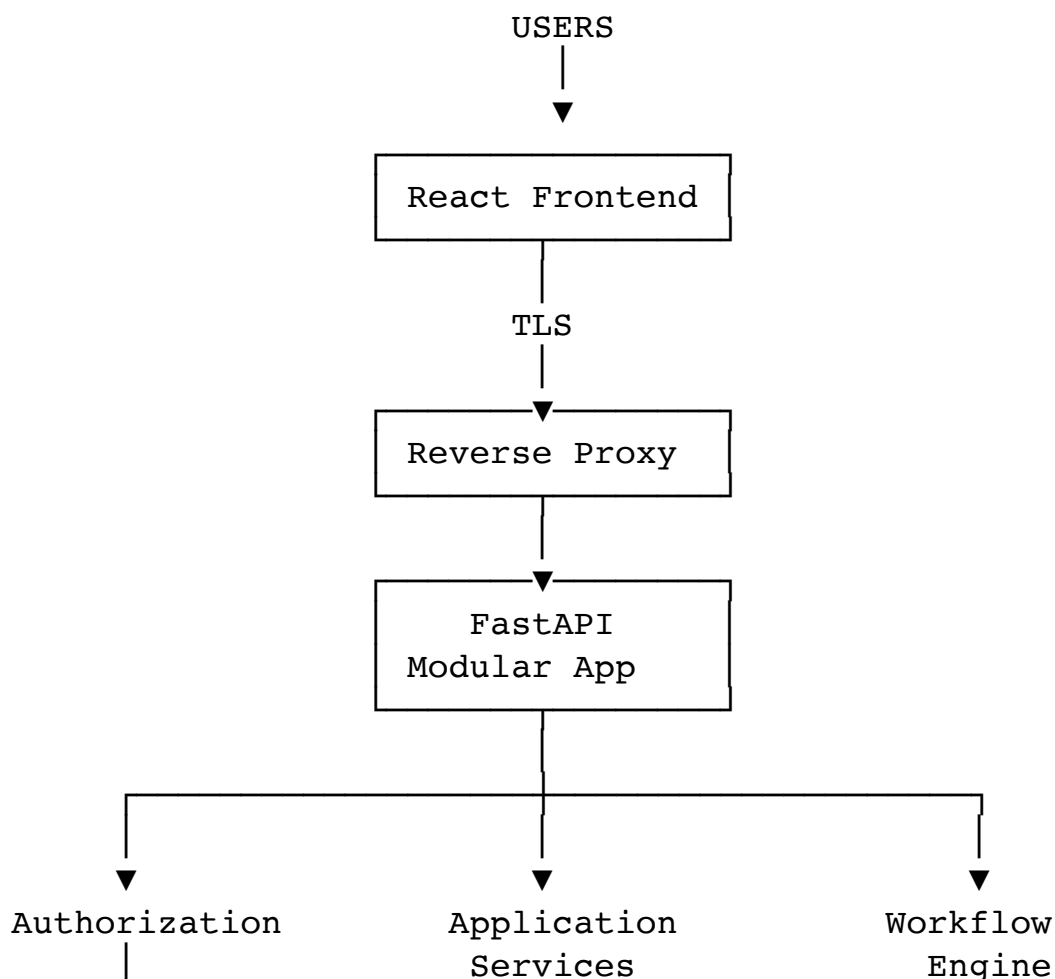
Reason: Prevent cross-case information leakage.

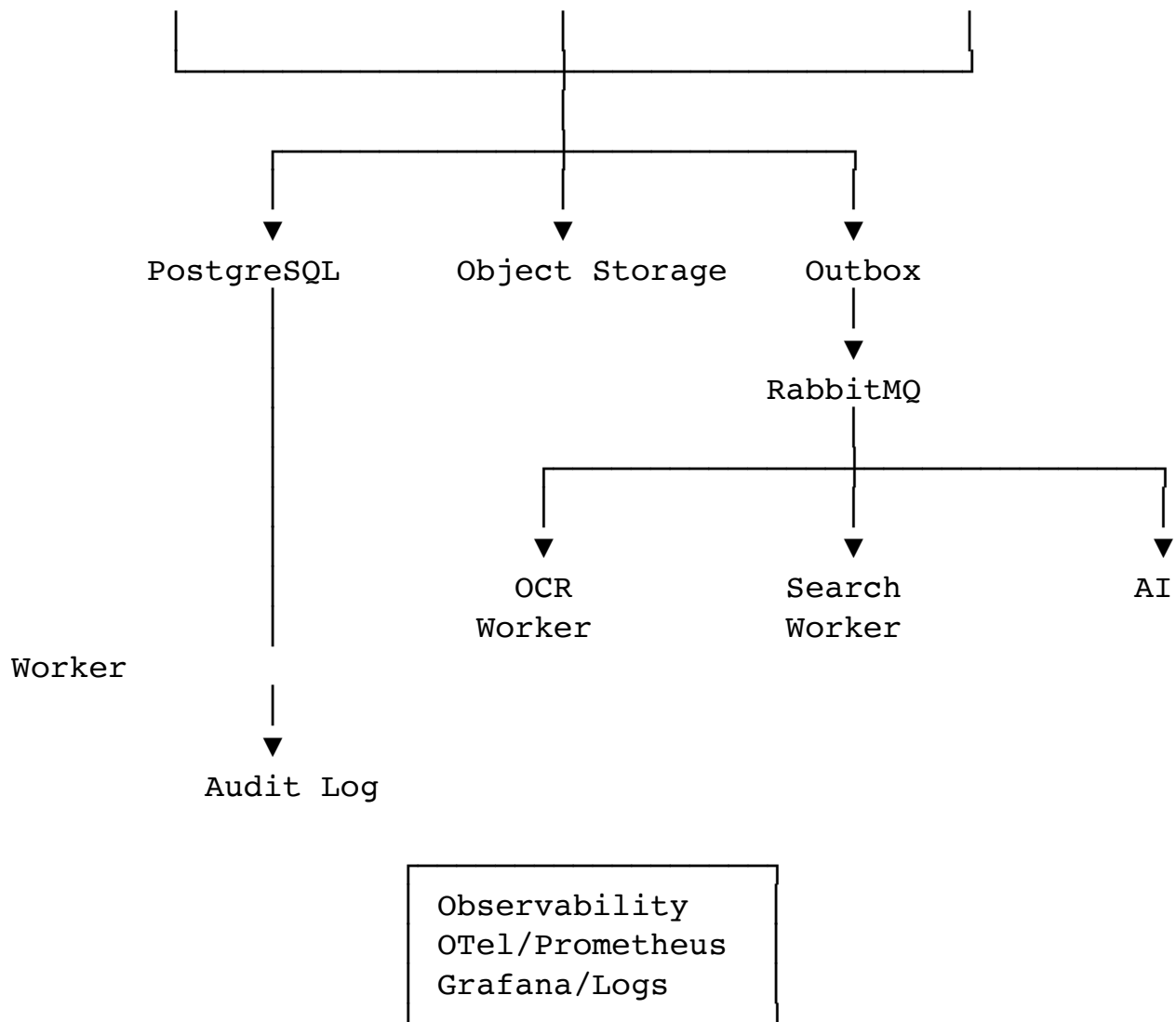
ADR-007 — Search

Decision: PostgreSQL FTS for MVP; OpenSearch as scalable extension.

Reason: Avoid unnecessary operational complexity in MVP.

81. Final Architecture





82. Definition of Done — v0.6

v0.6 telst lokið þegar eftirfarandi hefur verið ákveðið:

- Frontend architecture
- Backend architecture
- API layer
- Service layer
- Domain layer
- Repository layer
- Authentication
- Authorization
- Document processing

- Object storage
- Message queue
- Background workers
- Search architecture
- AI architecture
- Audit
- Observability
- CI/CD
- Containerization
- Kubernetes deployment model
- Disaster recovery targets
- Air-gapped deployment compatibility
- MVP scope
- Architecture decisions

83. Next Specification

Næsta eðlilega skref er:

Technical Specification v0.7

API Contract & OpenAPI Specification

Þar förum við frá architecture yfir í **nákvæmt API contract**.

v0.7 ætti að skilgreina:

```
POST    /api/v1/cases
GET     /api/v1/cases
GET     /api/v1/cases/{id}
PATCH  /api/v1/cases/{id}

POST    /api/v1/cases/{id}/transitions

POST    /api/v1/cases/{id}/documents
GET     /api/v1/documents/{id}
```

POST /api/v1/documents/{id}/versions

POST /api/v1/cases/{id}/evidence

POST /api/v1/cases/{id}/filings

POST /api/v1/cases/{id}/approvals

GET /api/v1/search

GET /api/v1/cases/{id}/audit

og fyrir **hvern endpoint**:

- HTTP method
- path
- authentication
- required permission
- request schema
- response schema
- HTTP status codes
- validation
- error codes
- audit event
- idempotency requirements
- pagination
- filtering
- sorting
- security considerations

Að lokum getum við búið til raunverulegt:

`openapi.yaml`

sem verður **machine-readable contract** milli React frontend, FastAPI backend og framtíðar integration-a.

Næsta skref, **v0.7**, verður því áhugavert: þá getum við farið að skrifa API-ið nánast eins og það ætti að birtast í Swagger/OpenAPI og þannig verið komin mjög nálægt fyrsta keyranlega prototype-inu.